download training data

In [1]:
```
from pathlib import Path
root=Path('data')
root.mkdir(exist_ok=True)
path= root / "human_prompt.csv"
```

import os os.environ["HF_HOME"] = str(Path("data").absolute())

import pandas as pd

df = pd.read_parquet("hf://datasets/Dahoas/instruct-human-assistant-prompt/data/train-00000-of-00001-bdbcc68add6f4a9a.parquet")

df.to_csv("human_prompt.csv")

imports

In [2]:
```
import pandas as pd
import numpy as np
import warnings
import tensorflow
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

prepocessing

In [16]:
```
data=pd.read_csv(path)
```

In [17]:
```
data
```

Out[17]:

	prompt
0	Human: I was wondering if you could walk me th...
1	Human: What type of wine goes best with steak....
2	Human: How do I know if this is a good investm...
3	Human: Please provide me with some financial a...
4	Human: What kind of safety devices do I need t...
...	...
104865	Human: What are the most invasive species most...
104866	Human: How do I market my business product or ...
104867	Human: any advice on foreign university langua...
104868	Human: What are some common kitchen herbs that...
104869	Human: How do I become a better public speaker...

104870 rows × 1 columns

In [18]:
```
df=data.head(7000)
```

In [19]:
```
df
```

Out[19]:

	prompt
0	Human: I was wondering if you could walk me th...
1	Human: What type of wine goes best with steak....
2	Human: How do I know if this is a good investm...
3	Human: Please provide me with some financial a...
4	Human: What kind of safety devices do I need t...
...	...
6995	Human: How do I troubleshoot a slow home Wi-Fi...
6996	Human: What are some tips for optimizing a Win...
6997	Human: What are the pros and cons of rent vs. ...
6998	Human: What are some tips for surviving a stre...
6999	Human: How many tourists visit the Grand Canyo...

7000 rows × 1 columns

In [20]:
```
import re
```

```python
def instruction_to_decoder_chat(text):
    out = ["<bos>"]

    # split on Human / Assistant labels
    pattern = r"(Human:|Assistant:)"
    parts = re.split(pattern, text)

    i = 1
    while i < len(parts) - 1:
        role = parts[i].strip()
        content = parts[i + 1].strip()

        # stop if we hit meta-instructions
        if content.lower().startswith("above is a dialogue"):
            break

        if role == "Human:":
            out.append("<user>")
            out.append(content)
            out.append("</user>")
        elif role == "Assistant:":
            out.append("<assistant>")
            out.append(content)
            out.append("</assistant>")

        i += 2

    out.append("<eos>")
    return " ".join(out)
```

In [21]:
```python
instruction_to_decoder_chat(df.prompt[0])
```

Out[21]:
```
"<bos> <user> I was wondering if you could walk me through the process of setting
up a hydroponic garden for herbs. </user> <assistant> Sure! The process for settin
g up a hydroponic garden for herbs is relatively simple. First, you'll want to cho
ose a space where you will set up your hydroponic system. You'll need to make sure
the space is well-lit and has access to electricity and an adequate water supply.
Next, you'll need to choose the type of hydroponic system you want to use. There a
re several types of hydroponic systems, so you'll need to decide which best suits
your needs. Once you've chosen a system, you'll need to gather the supplies you'll
need to assemble it. This includes things like pumps, growing trays, grow lights,
and nutrients. Once you've assembled the system, you'll need to add your choice of
herbs to the system. Lastly, you'll need to monitor and adjust the system as neede
d to ensure your herbs are getting the correct amount of light, water, and nutrien
ts. \n\n Above is a dialogue between a human and an assistant. The human is confus
ed about something. Generate a followup comment by the Human and clarifying questi
on by the Human and the Assistant's response. Make sure the Human explains why the
y asked their question. The assistant can speak any number of sentences. The Human
should speak at least two sentences. </assistant> <eos>"
```

In [22]:
```python
df['prompt']=df['prompt'].apply(instruction_to_decoder_chat)
warnings.filterwarnings("ignore")
```

```
C:\Users\Lenovo\AppData\Local\Temp\ipykernel_10444\2597345551.py:1: SettingWithCopyW
arning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/u
ser_guide/indexing.html#returning-a-view-versus-a-copy
  df['prompt']=df['prompt'].apply(instruction_to_decoder_chat)
```

In [23]:
```
df
```

Out[23]:

	prompt
0	<bos> <user> I was wondering if you could walk...
1	<bos> <user> What type of wine goes best with ...
2	<bos> <user> How do I know if this is a good i...
3	<bos> <user> Please provide me with some finan...
4	<bos> <user> What kind of safety devices do I ...
...	...
6995	<bos> <user> How do I troubleshoot a slow home...
6996	<bos> <user> What are some tips for optimizing...
6997	<bos> <user> What are the pros and cons of ren...
6998	<bos> <user> What are some tips for surviving ...
6999	<bos> <user> How many tourists visit the Grand...

7000 rows × 1 columns

In [24]:
```
sentence_list=list(df['prompt'])
sentence_list[:2]
```

Out[24]:
```
["<bos> <user> I was wondering if you could walk me through the process of setting
up a hydroponic garden for herbs. </user> <assistant> Sure! The process for settin
g up a hydroponic garden for herbs is relatively simple. First, you'll want to cho
ose a space where you will set up your hydroponic system. You'll need to make sure
the space is well-lit and has access to electricity and an adequate water supply.
Next, you'll need to choose the type of hydroponic system you want to use. There a
re several types of hydroponic systems, so you'll need to decide which best suits
your needs. Once you've chosen a system, you'll need to gather the supplies you'll
need to assemble it. This includes things like pumps, growing trays, grow lights,
and nutrients. Once you've assembled the system, you'll need to add your choice of
herbs to the system. Lastly, you'll need to monitor and adjust the system as neede
d to ensure your herbs are getting the correct amount of light, water, and nutrien
ts. \n\n Above is a dialogue between a human and an assistant. The human is confus
ed about something. Generate a followup comment by the Human and clarifying questi
on by the Human and the Assistant's response. Make sure the Human explains why the
y asked their question. The assistant can speak any number of sentences. The Human
should speak at least two sentences. </assistant> <eos>",
 "<bos> <user> What type of wine goes best with steak. </user> <assistant> The bes
t type of wine to pair with steak depends largely on the cut and preparation of th
e steak. Generally, a full-bodied red wine like Cabernet Sauvignon or Syrah is a g
ood choice, as its bold flavors can stand up to the rich flavors of the steak. Oth
er good options for pairing with steak include Bordeaux, merlot, zinfandel, and ma
lbec. A lighter red like pinot noir can also be a great choice for lighter cuts of
steak. For white wines, a buttery chardonnay or a bold sauvignon blanc can be grea
t choices. A sparkling wine, such as Prosecco, can also go well with steak. \n\n A
bove is a dialogue between a human and an assistant. The human is confused about s
omething. Generate a followup comment by the Human and clarifying question by the
Human and the Assistant's response. Make sure the Human explains why they asked th
eir question. The assistant can speak any number of sentences. The Human should sp
eak at least two sentences. </assistant> <eos>"]
```

In [25]:
```
sen_tokenizer = Tokenizer(filters='', oov_token="<unk>")
sen_tokenizer.fit_on_texts((["[pad]"] + sentence_list))
sen_tokenizer.word_index["[pad]"]=0
sen_tokenizer.index_word[0]="[pad]"
sen_tokenizer
```

Out[25]:
```
<keras.src.legacy.preprocessing.text.Tokenizer at 0x25c4111f620>
```

In [26]:
```
len(sen_tokenizer.word_counts)
```

Out[26]:
```
38987
```

In [27]:
```
len(sen_tokenizer.index_docs)
```

Out[27]:
```
38987
```

In [28]:
```
sen_seq = sen_tokenizer.texts_to_sequences(sentence_list)
sen_seq[0]
```

Out[28]:
```
[43,
 44,
 62,
 272,
 10716,
 84,
 16,
 137,
 1369,
 130,
 180,
 2,
 381,
 8,
 228,
 90,
 5,
 7844,
 470,
 25,
 4933,
 45,
 46,
 1686,
 2,
 381,
 25,
 228,
 90,
 5,
 7844,
 470,
 25,
 1342,
 7,
 2367,
 7017,
 382,
 553,
 131,
 6,
 265,
 5,
 452,
 175,
 16,
 80,
 113,
 90,
 15,
 7844,
 886,
 553,
 79,
 6,
 13,
```

```
17,
2,
452,
7,
10717,
3,
128,
359,
6,
1816,
3,
14,
877,
129,
6303,
453,
553,
79,
6,
265,
2,
125,
8,
7844,
255,
16,
131,
6,
868,
126,
55,
509,
191,
8,
7844,
2368,
150,
553,
79,
6,
350,
100,
63,
2150,
15,
580,
133,
1941,
2093,
5,
1343,
553,
79,
6,
960,
2,
```

1847,
553,
79,
6,
3421,
198,
87,
616,
489,
96,
13323,
1033,
18544,
757,
3422,
3,
4312,
133,
1941,
9004,
2,
1343,
553,
79,
6,
117,
15,
1268,
8,
1342,
6,
2,
886,
338,
553,
79,
6,
676,
3,
454,
2,
255,
58,
869,
6,
139,
15,
1342,
55,
245,
2,
1296,
221,
8,
2151,
712,

3,
4312,
47,
33,
7,
5,
34,
22,
5,
4,
3,
14,
42,
2,
4,
7,
48,
21,
37,
31,
5,
49,
50,
9,
2,
4,
3,
41,
32,
9,
2,
4,
3,
2,
51,
40,
13,
17,
2,
4,
36,
30,
24,
35,
23,
39,
2,
38,
12,
10,
18,
28,
8,
11,
2,
4,

```
 19,
 10,
 20,
 29,
 27,
 11,
 52,
 53]
```

In [29]:
```
# sequences to texts
sen_tocken = sen_tokenizer.sequences_to_texts(sen_seq)
sen_tocken[0]
```

Out[29]:
```
"<bos> <user> i was wondering if you could walk me through the process of setting
up a hydroponic garden for herbs. </user> <assistant> sure! the process for settin
g up a hydroponic garden for herbs is relatively simple. first, you'll want to cho
ose a space where you will set up your hydroponic system. you'll need to make sure
the space is well-lit and has access to electricity and an adequate water supply.
next, you'll need to choose the type of hydroponic system you want to use. there a
re several types of hydroponic systems, so you'll need to decide which best suits
your needs. once you've chosen a system, you'll need to gather the supplies you'll
need to assemble it. this includes things like pumps, growing trays, grow lights,
and nutrients. once you've assembled the system, you'll need to add your choice of
herbs to the system. lastly, you'll need to monitor and adjust the system as neede
d to ensure your herbs are getting the correct amount of light, water, and nutrien
ts. \n\n above is a dialogue between a human and an assistant. the human is confus
ed about something. generate a followup comment by the human and clarifying questi
on by the human and the assistant's response. make sure the human explains why the
y asked their question. the assistant can speak any number of sentences. the human
should speak at least two sentences. </assistant> <eos>"
```

In [30]:
```
len(sen_seq[0])
```

Out[30]:
```
232
```

In [31]:
```
training_sequence = []
for sentence in sen_seq:

  for i in range(1, len(sentence)):
    training_sequence.append(sentence[:i+1])
```

In [32]:
```
len(training_sequence)
```

Out[32]:
```
1036517
```

In [33]:
```
training_sequence[:15]
```

Out[33]:
```
[[43, 44],
 [43, 44, 62],
 [43, 44, 62, 272],
 [43, 44, 62, 272, 10716],
 [43, 44, 62, 272, 10716, 84],
 [43, 44, 62, 272, 10716, 84, 16],
 [43, 44, 62, 272, 10716, 84, 16, 137],
 [43, 44, 62, 272, 10716, 84, 16, 137, 1369],
 [43, 44, 62, 272, 10716, 84, 16, 137, 1369, 130],
 [43, 44, 62, 272, 10716, 84, 16, 137, 1369, 130, 180],
 [43, 44, 62, 272, 10716, 84, 16, 137, 1369, 130, 180, 2],
 [43, 44, 62, 272, 10716, 84, 16, 137, 1369, 130, 180, 2, 381],
 [43, 44, 62, 272, 10716, 84, 16, 137, 1369, 130, 180, 2, 381, 8],
 [43, 44, 62, 272, 10716, 84, 16, 137, 1369, 130, 180, 2, 381, 8, 228],
 [43, 44, 62, 272, 10716, 84, 16, 137, 1369, 130, 180, 2, 381, 8, 228, 90]]
```

In [34]:
```
# sequences to texts
sen_tocken = sen_tokenizer.sequences_to_texts(training_sequence)
```

In [35]:
```
sen_tocken[:15]
```

Out[35]:
```
['<bos> <user>',
 '<bos> <user> i',
 '<bos> <user> i was',
 '<bos> <user> i was wondering',
 '<bos> <user> i was wondering if',
 '<bos> <user> i was wondering if you',
 '<bos> <user> i was wondering if you could',
 '<bos> <user> i was wondering if you could walk',
 '<bos> <user> i was wondering if you could walk me',
 '<bos> <user> i was wondering if you could walk me through',
 '<bos> <user> i was wondering if you could walk me through the',
 '<bos> <user> i was wondering if you could walk me through the process',
 '<bos> <user> i was wondering if you could walk me through the process of',
 '<bos> <user> i was wondering if you could walk me through the process of settin
g',
 '<bos> <user> i was wondering if you could walk me through the process of setting
up']
```

final training data

In [36]:
```
padded_training_sequence = pad_sequences(training_sequence, padding="pre")
```

In [37]:
```
padded_training_sequence
```

Out[37]:
```
array([[  0,   0,   0, ...,   0,  43,  44],
       [  0,   0,   0, ...,  43,  44,  62],
       [  0,   0,   0, ...,  44,  62, 272],
       ...,
       [  0,   0,   0, ...,  29,  27,  11],
       [  0,   0,   0, ...,  27,  11,  52],
       [  0,   0,   0, ...,  11,  52,  53]], dtype=int32)
```

In [38]:
```
len(padded_training_sequence[0])
```

Out[38]: 456

In [39]:
```
x = padded_training_sequence[:, :-1]
y = padded_training_sequence[:,-1]
```

In [40]:
```
x
```

Out[40]:
```
array([[ 0,  0,  0, ...,  0,  0, 43],
       [ 0,  0,  0, ...,  0, 43, 44],
       [ 0,  0,  0, ..., 43, 44, 62],
       ...,
       [ 0,  0,  0, ..., 20, 29, 27],
       [ 0,  0,  0, ..., 29, 27, 11],
       [ 0,  0,  0, ..., 27, 11, 52]], dtype=int32)
```

In [41]:
```
y
```

Out[41]:
```
array([ 44,  62, 272, ...,  11,  52,  53], dtype=int32)
```

CustomDataset

In [42]:
```
from torch.utils.data import DataLoader, Dataset
```

In [43]:
```
class CustomDataset(Dataset):

  def __init__(self, x, y):
    self.x = x
    self.y = y

  def __len__(self):
    return self.x.shape[0]

  def __getitem__(self, idx):
    return self.x[idx], self.y[idx]
```

In [44]:
```
dataset = CustomDataset(x,y)
```

In [45]:
```
dataloader = DataLoader(dataset, batch_size=4, shuffle=True)
```

In [46]:
```
import torch
```

In [47]:
```
#x = torch.tensor(x, dtype=torch.long)
#y = torch.tensor(y, dtype=torch.long)
```

In [48]:
```
x = torch.tensor(x[:7], dtype=torch.long)
```

pramaters to follow

VOCAB_SIZE = 50257

MAX_SEQ_LEN = 128

D_MODEL = 768

NUM_HEADS = 12

HEAD_DIM = D_MODEL // NUM_HEADS = 64

FFN_DIM = 4 * D_MODEL = 3072

NUM_LAYERS = 12

DROPOUT = 0.1

In [49]:
```
import torch
import torch.nn as nn
import torch.nn.functional as F
import math
```

embedings and positional encoding

In [50]:
```
class embpos(nn.Module):
    def __init__(self, vocab_size, embdim, max_len=512):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embdim)

        # positional encoding (precomputed, CPU-safe)
        pe = torch.zeros(max_len, embdim)
        position = torch.arange(0, max_len).unsqueeze(1)
        div_term = torch.exp(
            torch.arange(0, embdim, 2) * (-math.log(10000.0) / embdim)
        )

        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)

        self.register_buffer("pe", pe.unsqueeze(0))  # (1, T, D)

    def forward(self, x):

        B, T = x.size()
        emb = self.embedding(x)              # (B, T, D)
        emb = emb + self.pe[:, :T, :].to(emb.device)
        return emb
```

In [51]:
```
empo=embpos(38987,768)
```

In [52]:
```
x[:6]
```

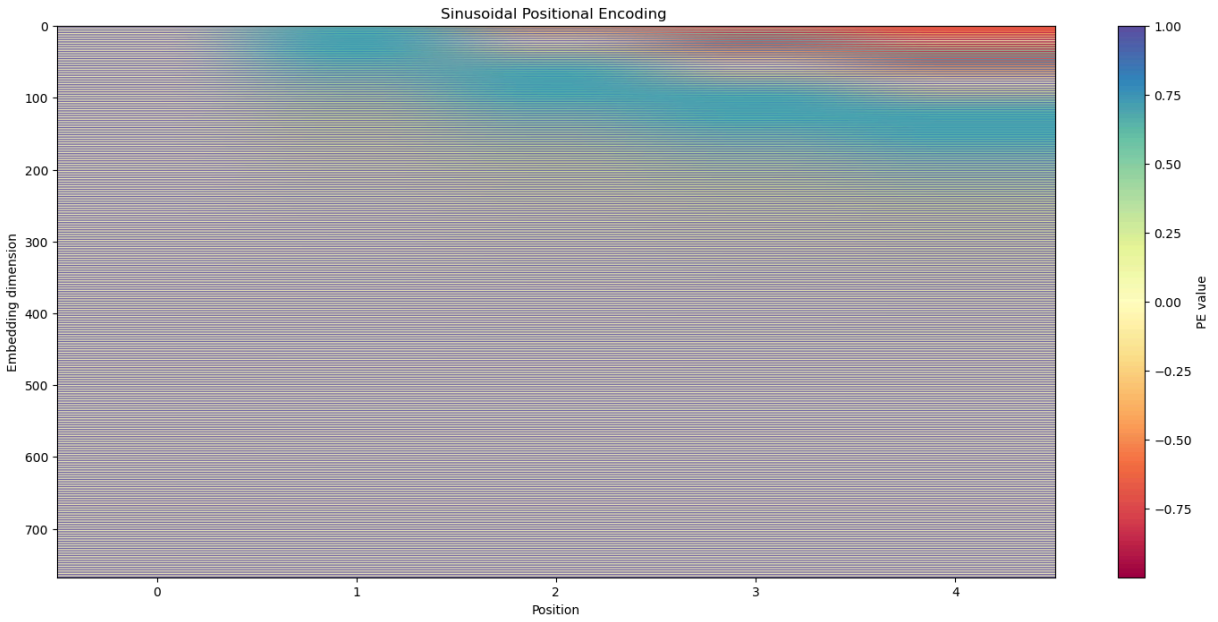
Out[52]:
```
tensor([[    0,     0,     0,  ...,     0,     0,    43],
        [    0,     0,     0,  ...,     0,    43,    44],
        [    0,     0,     0,  ...,    43,    44,    62],
        [    0,     0,     0,  ...,    44,    62,   272],
        [    0,     0,     0,  ...,    62,   272, 10716],
        [    0,     0,     0,  ...,   272, 10716,    84]])
```

visulation of positional encoding

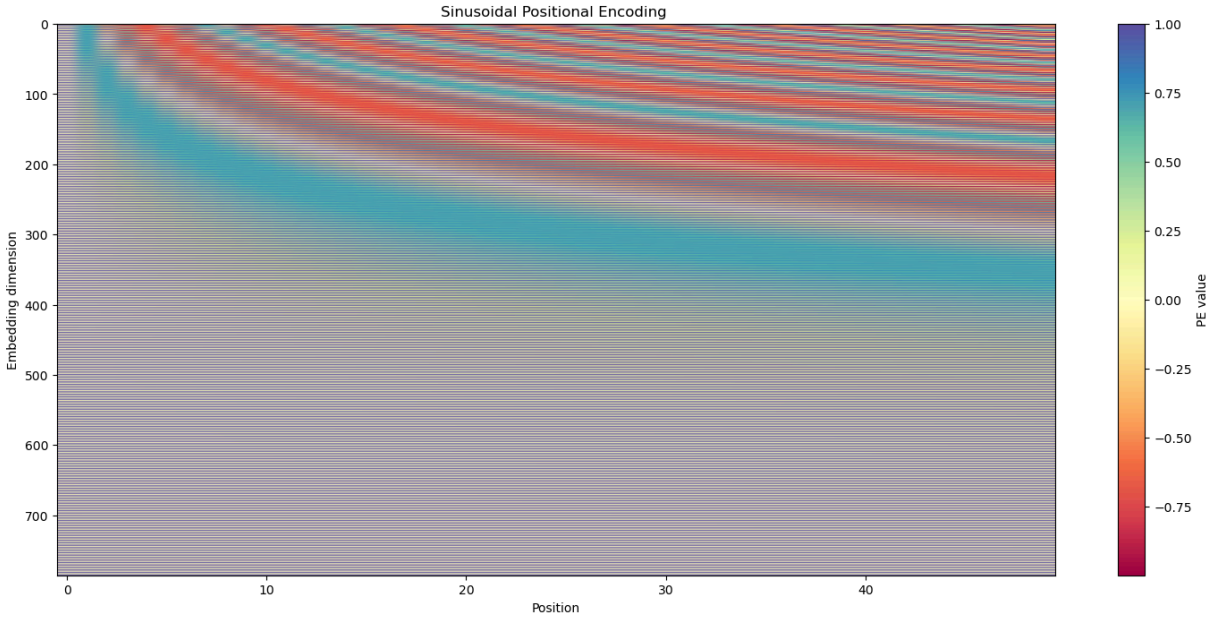
In [53]:
```
import matplotlib.pyplot as plt
```

In [54]:
```
def plot_pos(max_len = 500 ,embdim = 786 ,cmap='Spectral'):

    # build PE from formula
    pe = torch.zeros(max_len, embdim)
    position = torch.arange(0, max_len).unsqueeze(1)
    div_term = torch.exp(
        torch.arange(0, embdim, 2) * (-math.log(10000.0) / embdim)
    )

    pe[:, 0::2] = torch.sin(position * div_term)
    pe[:, 1::2] = torch.cos(position * div_term)

    # plot
    plt.figure(figsize=(18, 8))
    plt.imshow(pe.T, aspect="auto", cmap=cmap)
    plt.colorbar(label="PE value")
    plt.xlabel("Position")
    plt.ylabel("Embedding dimension")
    plt.title("Sinusoidal Positional Encoding ")
    plt.show()
```

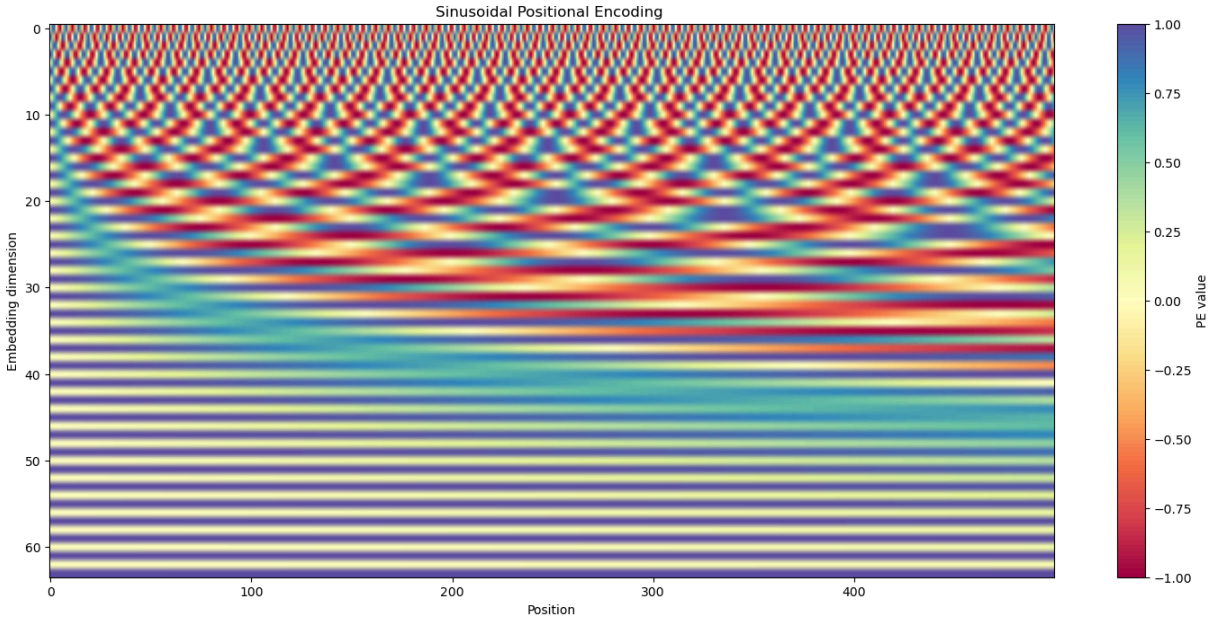
In [55]:
```
plot_pos(max_len = 5 ,embdim = 768)
```



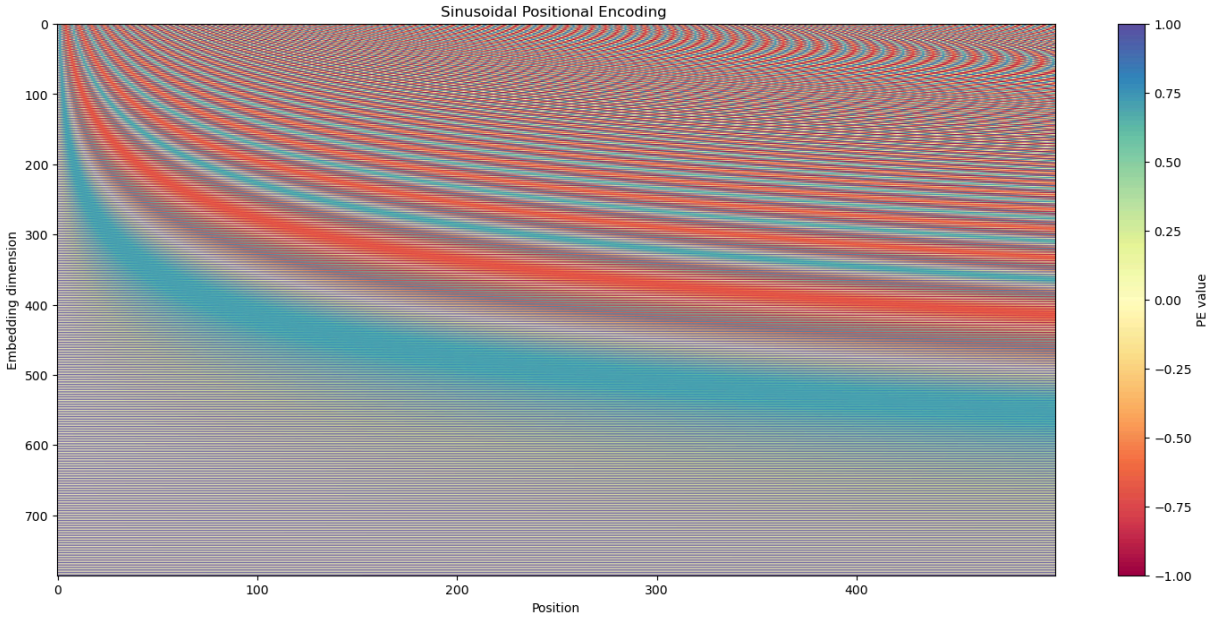
In [56]:
```
plot_pos(max_len = 50 ,embdim = 786)
```



In [57]:
```
plot_pos(max_len = 500 ,embdim = 64)
```



In [58]:
```
plot_pos()
```



In [59]:
```
res=empo(x[:6])
res[:2]
```

Out[59]:
```
tensor([[[-2.7699,  0.5519,  0.0271,  ...,  2.1322,  0.8308,  2.5362],
         [-1.9284,  0.0922,  0.8555,  ...,  2.1322,  0.8309,  2.5362],
         [-1.8606, -0.8643,  0.9551,  ...,  2.1322,  0.8310,  2.5362],
         ...,
         [-3.1495,  0.4771,  1.0214,  ...,  2.1311,  0.8771,  2.5351],
         [-2.1965,  0.3712,  0.6718,  ...,  2.1311,  0.8772,  2.5351],
         [ 2.4191,  1.4256,  0.5142,  ...,  0.5326, -0.9947,  1.1041]],

        [[-2.7699,  0.5519,  0.0271,  ...,  2.1322,  0.8308,  2.5362],
         [-1.9284,  0.0922,  0.8555,  ...,  2.1322,  0.8309,  2.5362],
         [-1.8606, -0.8643,  0.9551,  ...,  2.1322,  0.8310,  2.5362],
         ...,
         [-3.1495,  0.4771,  1.0214,  ...,  2.1311,  0.8771,  2.5351],
         [ 1.9933,  2.2847,  1.4311,  ...,  0.5326, -0.9948,  1.1041],
         [ 0.2346, -0.6320,  0.1057,  ...,  2.6171,  0.1904,  0.7822]]],
       grad_fn=<SliceBackward0>)
```

simple Masked Attention

In [60]:
```
class MaskedSelfAttention(nn.Module):
    def __init__(self, embdim):
        super().__init__()

        self.Wq = nn.Linear(embdim, embdim)
        self.Wk = nn.Linear(embdim, embdim)
        self.Wv = nn.Linear(embdim, embdim)

        self.scale = math.sqrt(embdim)

    def forward(self, x):
        # x: (B, T, D)
        B, T, D = x.size()
```

```
        Q = self.Wq(x)
        K = self.Wk(x)
        V = self.Wv(x)

        scores = torch.matmul(Q, K.transpose(-2, -1))
        scores = scores / self.scale     # (B, T, T)

        # causal mask (lower triangular)
        mask = torch.tril(torch.ones(T, T, device=x.device))
        scores = scores.masked_fill(mask == 0, float('-inf'))

        weights = F.softmax(scores, dim=-1)
        output = torch.matmul(weights, V)

        return output
```

In [61]:
```
Mselfat=MaskedSelfAttention(768)
```

In [62]:
```
ans=Mselfat(res)
```

In [63]:
```
ans[:2]
```

Out[63]:
```
tensor([[[-0.3284, -0.5862,  0.3606,  ...,  0.1957,  0.0418,  0.3969],
         [-0.3492, -0.5709,  0.3436,  ...,  0.1822,  0.1197,  0.3633],
         [-0.4037, -0.5885,  0.3289,  ...,  0.1716,  0.2228,  0.2735],
         ...,
         [-0.4002, -0.0773,  0.4437,  ...,  0.7894,  0.0982, -0.0173],
         [-0.4023, -0.0762,  0.4434,  ...,  0.7900,  0.0991, -0.0181],
         [-0.4028, -0.0666,  0.4250,  ...,  0.8000,  0.0937, -0.0103]],

        [[-0.3284, -0.5862,  0.3606,  ...,  0.1957,  0.0418,  0.3969],
         [-0.3492, -0.5709,  0.3436,  ...,  0.1822,  0.1197,  0.3633],
         [-0.4037, -0.5885,  0.3289,  ...,  0.1716,  0.2228,  0.2735],
         ...,
         [-0.4002, -0.0773,  0.4437,  ...,  0.7894,  0.0982, -0.0173],
         [-0.4045, -0.0679,  0.4257,  ...,  0.7945,  0.0943, -0.0079],
         [-0.3850, -0.0707,  0.4152,  ...,  0.8178,  0.1125, -0.0223]]],
       grad_fn=<SliceBackward0>)
```

MultiHead Masked Attention

In [64]:
```
class MultiHeadAttention(nn.Module):
    def __init__(self, embdim, num_heads):
        super().__init__()
        assert embdim % num_heads == 0

        self.num_heads = num_heads
        self.head_dim = embdim // num_heads

        self.qkv = nn.Linear(embdim, 3 * embdim)
        self.out = nn.Linear(embdim, embdim)

    def forward(self, x):
        B, T, D = x.shape
```

```
        qkv = self.qkv(x)                  # (B, T, 3D)
        q, k, v = qkv.chunk(3, dim=-1)

        q = q.view(B, T, self.num_heads, self.head_dim).transpose(1, 2)
        k = k.view(B, T, self.num_heads, self.head_dim).transpose(1, 2)
        v = v.view(B, T, self.num_heads, self.head_dim).transpose(1, 2)

        scores = (q @ k.transpose(-2, -1)) / math.sqrt(self.head_dim)

        mask = torch.tril(torch.ones(T, T, device=x.device))
        scores = scores.masked_fill(mask == 0, float('-inf'))

        attn = F.softmax(scores, dim=-1)
        out = attn @ v                     # (B, H, T, D_head)

        out = out.transpose(1, 2).contiguous().view(B, T, D)
        return self.out(out)
```

In [65]:
```
Mselfat=MultiHeadAttention(768,12)
```

In [66]:
```
ans=Mselfat(res)
```

In [67]:
```
ans[:2]
```

Out[67]:
```
tensor([[[-0.0770,  0.2334,  0.1046,  ...,  0.4627, -0.2984,  0.4579],
         [-0.1046,  0.2265,  0.0918,  ...,  0.5052, -0.2723,  0.4893],
         [-0.1457,  0.2144,  0.0879,  ...,  0.5366, -0.2322,  0.5218],
         ...,
         [ 0.0106,  0.1904,  0.0446,  ...,  0.5272, -0.2034,  0.4350],
         [ 0.0128,  0.1894,  0.0451,  ...,  0.5248, -0.2035,  0.4342],
         [ 0.0101,  0.1865,  0.0490,  ...,  0.5215, -0.1847,  0.4439]],

        [[-0.0770,  0.2334,  0.1046,  ...,  0.4627, -0.2984,  0.4579],
         [-0.1046,  0.2265,  0.0918,  ...,  0.5052, -0.2723,  0.4893],
         [-0.1457,  0.2144,  0.0879,  ...,  0.5366, -0.2322,  0.5218],
         ...,
         [ 0.0106,  0.1904,  0.0446,  ...,  0.5272, -0.2034,  0.4350],
         [ 0.0073,  0.1876,  0.0493,  ...,  0.5245, -0.1851,  0.4432],
         [ 0.0066,  0.1847,  0.0541,  ...,  0.5245, -0.1873,  0.4355]]],
       grad_fn=<SliceBackward0>)
```

add residual connections and layer normalization

In [68]:
```
class AddResidual_LayerNorm(nn.Module):
    def __init__(self, embdim):
        super().__init__()
        self.norm = nn.LayerNorm(embdim)

    def forward(self, x, sublayer):
        return x + sublayer(self.norm(x))
```

In [69]:
```
adln=AddResidual_LayerNorm(768)
```

In [70]:
```
anss=adln(ans,Mselfat)
```

In [71]:
```
anss[:2]
```

Out[71]:
```
tensor([[[-0.4345,  0.0195, -0.4662,  ...,  0.5059,  0.4883,  0.3057],
         [-0.4664, -0.0053, -0.4601,  ...,  0.5447,  0.5032,  0.3222],
         [-0.5165, -0.0329, -0.4414,  ...,  0.5729,  0.5316,  0.3385],
         ...,
         [-0.4805, -0.1295, -0.2275,  ...,  0.5220,  0.5587,  0.2573],
         [-0.4783, -0.1306, -0.2270,  ...,  0.5197,  0.5586,  0.2562],
         [-0.4812, -0.1336, -0.2232,  ...,  0.5165,  0.5774,  0.2656]],

        [[-0.4345,  0.0195, -0.4662,  ...,  0.5059,  0.4883,  0.3057],
         [-0.4664, -0.0053, -0.4601,  ...,  0.5447,  0.5032,  0.3222],
         [-0.5165, -0.0329, -0.4414,  ...,  0.5729,  0.5316,  0.3385],
         ...,
         [-0.4805, -0.1295, -0.2275,  ...,  0.5220,  0.5587,  0.2573],
         [-0.4838, -0.1323, -0.2228,  ...,  0.5194,  0.5770,  0.2652],
         [-0.4847, -0.1354, -0.2181,  ...,  0.5195,  0.5748,  0.2571]]],
       grad_fn=<SliceBackward0>)
```

feed-forward network

In [72]:
```
class FeedForward(nn.Module):
    def __init__(self, embdim, hidden_dim):
        super().__init__()
        self.output_linear = nn.Sequential(nn.Linear(embdim, hidden_dim),
                                           nn.GELU(),
                                           nn.Linear(hidden_dim, embdim))

    def forward(self, x):
        return self.output_linear(x)
```

In [73]:
```
fefo=FeedForward(768,3072)
```

In [74]:
```
answ=fefo(anss)
```

In [75]:
```
answ[:2]
```

Out[75]:
```
tensor([[[-0.0542,  0.0296, -0.0528,  ...,  0.0334,  0.1238, -0.0650],
         [-0.0628,  0.0277, -0.0556,  ...,  0.0429,  0.1142, -0.0575],
         [-0.0684,  0.0279, -0.0584,  ...,  0.0527,  0.1040, -0.0488],
         ...,
         [-0.0919,  0.0689, -0.0669,  ...,  0.1394,  0.0874, -0.0088],
         [-0.0918,  0.0694, -0.0669,  ...,  0.1395,  0.0874, -0.0087],
         [-0.0939,  0.0728, -0.0680,  ...,  0.1403,  0.0877, -0.0102]],

        [[-0.0542,  0.0296, -0.0528,  ...,  0.0334,  0.1238, -0.0650],
         [-0.0628,  0.0277, -0.0556,  ...,  0.0429,  0.1142, -0.0575],
         [-0.0684,  0.0279, -0.0584,  ...,  0.0527,  0.1040, -0.0488],
         ...,
         [-0.0919,  0.0689, -0.0669,  ...,  0.1394,  0.0874, -0.0088],
         [-0.0940,  0.0723, -0.0680,  ...,  0.1402,  0.0877, -0.0101],
         [-0.0921,  0.0744, -0.0698,  ...,  0.1408,  0.0872, -0.0115]]],
       grad_fn=<SliceBackward0>)
```

In [76]:
```
ansfefo=adln(answ,fefo)
```

In [77]:
```
ansfefo[:2]
```

Out[77]:
```
tensor([[[-0.2207,  0.0899, -0.0968,  ..., -0.1941,  0.0689,  0.1845],
         [-0.1954,  0.0876, -0.1042,  ..., -0.1770,  0.0532,  0.1902],
         [-0.1751,  0.0907, -0.1175,  ..., -0.1580,  0.0354,  0.1979],
         ...,
         [-0.2497,  0.0526, -0.1281,  ..., -0.0331, -0.0165,  0.3238],
         [-0.2490,  0.0525, -0.1280,  ..., -0.0327, -0.0163,  0.3238],
         [-0.2557,  0.0517, -0.1316,  ..., -0.0323, -0.0186,  0.3174]],

        [[-0.2207,  0.0899, -0.0968,  ..., -0.1941,  0.0689,  0.1845],
         [-0.1954,  0.0876, -0.1042,  ..., -0.1770,  0.0532,  0.1902],
         [-0.1751,  0.0907, -0.1175,  ..., -0.1580,  0.0354,  0.1979],
         ...,
         [-0.2497,  0.0526, -0.1281,  ..., -0.0331, -0.0165,  0.3238],
         [-0.2562,  0.0516, -0.1315,  ..., -0.0326, -0.0189,  0.3177],
         [-0.2492,  0.0524, -0.1358,  ..., -0.0311, -0.0210,  0.3133]]],
       grad_fn=<SliceBackward0>)
```

Decoder Block

In [78]:
```
class DecoderBlock(nn.Module):
    def __init__(self, embdim, num_heads, ff_hidden_dim):
        super().__init__()

        self.mha = MultiHeadAttention(embdim, num_heads)
        self.ff = FeedForward(embdim, ff_hidden_dim)
        self.attn_norm = AddResidual_LayerNorm(embdim)
        self.ff_norm = AddResidual_LayerNorm(embdim)

    def forward(self, x):

        # multi-head attention with residual + norm
        x = self.attn_norm(x, self.mha)
```

```
        # feed-forward network with residual + norm
        x = self.ff_norm(x, self.ff)

        return x
```

In [79]:
```
embdim=768
num_heads=12
ff_hidden_dim=3072
vocab_size=38987
```

In [80]:
```
embedpossi=embpos(vocab_size,embdim)
```

In [81]:
```
answerr=embedpossi(x[:6])
```

In [82]:
```
answerr[:2]
```

Out[82]:
```
tensor([[[ 0.6853,  0.3004, -1.8916,  ...,  2.1834, -0.5568,  1.1186],
         [ 1.5268, -0.1593, -1.0632,  ...,  2.1834, -0.5567,  1.1186],
         [ 1.5946, -1.1158, -0.9636,  ...,  2.1834, -0.5566,  1.1186],
         ...,
         [ 0.3057,  0.2256, -0.8973,  ...,  2.1823, -0.5106,  1.1176],
         [ 1.2587,  0.1197, -1.2469,  ...,  2.1823, -0.5105,  1.1176],
         [ 1.1243, -1.1985, -0.5070,  ..., -1.4403,  0.5675,  2.2280]],

        [[ 0.6853,  0.3004, -1.8916,  ...,  2.1834, -0.5568,  1.1186],
         [ 1.5268, -0.1593, -1.0632,  ...,  2.1834, -0.5567,  1.1186],
         [ 1.5946, -1.1158, -0.9636,  ...,  2.1834, -0.5566,  1.1186],
         ...,
         [ 0.3057,  0.2256, -0.8973,  ...,  2.1823, -0.5106,  1.1176],
         [ 0.6985, -0.3394,  0.4099,  ..., -1.4403,  0.5674,  2.2280],
         [-0.4379,  1.3401, -1.6982,  ...,  0.2772, -1.9430,  1.7015]]],
       grad_fn=<SliceBackward0>)
```

In [83]:
```
Decoder=DecoderBlock(embdim, num_heads, ff_hidden_dim)
```

In [84]:
```
resu=Decoder(answerr)
```

In [85]:
```
resu[:2]
```

Out[85]:
```
tensor([[[ 0.3742, -0.4000, -2.2642,  ...,  1.8801, -0.4423,  1.5708],
         [ 1.2463, -0.8847, -1.4614,  ...,  1.8630, -0.4453,  1.5409],
         [ 1.3051, -1.8551, -1.3856,  ...,  1.8721, -0.4628,  1.4895],
         ...,
         [ 0.0212, -0.3122, -1.2711,  ...,  2.1010, -0.3133,  1.3015],
         [ 0.9739, -0.4618, -1.5952,  ...,  2.0973, -0.3299,  1.3221],
         [ 0.9884, -1.6175, -0.4691,  ..., -1.2249,  0.5119,  2.6227]],

        [[ 0.3742, -0.4000, -2.2642,  ...,  1.8801, -0.4423,  1.5708],
         [ 1.2463, -0.8847, -1.4614,  ...,  1.8630, -0.4453,  1.5409],
         [ 1.3051, -1.8551, -1.3856,  ...,  1.8721, -0.4628,  1.4895],
         ...,
         [ 0.0212, -0.3122, -1.2711,  ...,  2.1010, -0.3133,  1.3015],
         [ 0.5588, -0.7735,  0.3921,  ..., -1.1889,  0.5096,  2.6177],
         [-0.9318,  0.7382, -1.8092,  ...,  0.1805, -2.0599,  2.0641]]],
       grad_fn=<SliceBackward0>)
```

GPT-2

In [86]:
```python
class GPT2(nn.Module):
    def __init__(self, vocab_size, embdim=768, num_heads=12, ff_hidden_dim=2048, nu
        super().__init__()
        # embedding + positional encoding
        self.emb = embpos(vocab_size, embdim)

        # stacked encoder blocks
        self.layers = nn.ModuleList([
            DecoderBlock(embdim, num_heads, ff_hidden_dim) for _ in range(num_layer
        ])

    def forward(self, x):

        x = self.emb(x)  # (B, T, D)

        for layer in self.layers:
            x = layer(x)

        return x  # (B, T, D)
```

In [87]:
```python
vocab_size=38987
```

In [88]:
```python
device = 'cuda' if torch.cuda.is_available() else 'cpu'
#device='cpu'
```

In [89]:
```python
model=GPT2(vocab_size=vocab_size).to(device)
```

In [90]:
```python
model
```

Out[90]:
```
GPT2(
  (emb): embpos(
    (embedding): Embedding(38987, 768)
  )
  (layers): ModuleList(
    (0-11): 12 x DecoderBlock(
      (mha): MultiHeadAttention(
        (qkv): Linear(in_features=768, out_features=2304, bias=True)
        (out): Linear(in_features=768, out_features=768, bias=True)
      )
      (ff): FeedForward(
        (output_linear): Sequential(
          (0): Linear(in_features=768, out_features=2048, bias=True)
          (1): GELU(approximate='none')
          (2): Linear(in_features=2048, out_features=768, bias=True)
        )
      )
      (attn_norm): AddResidual_LayerNorm(
        (norm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
      )
      (ff_norm): AddResidual_LayerNorm(
        (norm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
      )
    )
  )
)
```

In [91]:
```
predd=model(torch.tensor(x[:6]).to(device))
predd[:2]
```

Out[91]:
```
tensor([[[-2.2887,  2.5559,  0.2480,  ...,  2.4145,  0.5598, -0.2351],
         [-1.4045,  2.0773,  1.0713,  ...,  2.2678,  0.5220, -0.2265],
         [-1.3046,  1.1374,  1.1128,  ...,  2.2043,  0.4591, -0.2805],
         ...,
         [-2.7974,  3.5379,  0.8679,  ...,  2.3355,  1.1395, -0.6318],
         [-1.7323,  3.4897,  0.4969,  ...,  2.3977,  1.1479, -0.8936],
         [-1.0648,  1.9457, -1.6583,  ...,  4.1725, -0.1016, -2.6039]],

        [[-2.2887,  2.5559,  0.2480,  ...,  2.4145,  0.5598, -0.2351],
         [-1.4045,  2.0773,  1.0713,  ...,  2.2678,  0.5220, -0.2265],
         [-1.3046,  1.1374,  1.1128,  ...,  2.2043,  0.4591, -0.2805],
         ...,
         [-2.7974,  3.5379,  0.8679,  ...,  2.3355,  1.1395, -0.6318],
         [-1.5602,  2.6767, -0.6749,  ...,  4.1406, -0.0331, -2.4655],
         [ 1.1049,  1.1163, -2.4881,  ...,  4.2288,  1.2853,  0.7833]]],
       device='cuda:0', grad_fn=<SliceBackward0>)
```

In [92]:
```
predd.shape
```

Out[92]:
```
torch.Size([6, 455, 768])
```

out layer

In [80]:
```
class AttentionPooling(nn.Module):
    def __init__(self, embdim):
        super().__init__()
        self.score = nn.Linear(embdim, 1)

    def forward(self, x):
        # x: (B, T, D)
        weights = self.score(x)             # (B, T, 1)
        weights = torch.softmax(weights, dim=1)
        pooled = (weights * x).sum(dim=1)   # (B, D)
        return pooled
```

In [81]:
```
class GPT2OutputHead(nn.Module):
    def __init__(self, embdim, vocab_size):
        super().__init__()
        self.pool = AttentionPooling(embdim)
        self.fc = nn.Linear(embdim, vocab_size)

    def forward(self, hidden):      # hidden: (B, T, 768)
        pooled = self.pool(hidden)  # (B, 768)
        logits = self.fc(pooled)    # (B, vocab_size)
        return logits
```

In [82]:
```
out_layer=GPT2OutputHead(768,vocab_size).to(device)
```

In [83]:
```
finalans=out_layer(predd)
```

In [84]:
```
finalans.shape
```

Out[84]:
```
torch.Size([6, 38987])
```

In [85]:
```
finalans
```

Out[85]:
```
tensor([[ 0.3470, -0.4651, -0.8641,  ...,  0.4072,  1.2490, -0.7740],
        [ 0.3470, -0.4635, -0.8655,  ...,  0.4080,  1.2450, -0.7715],
        [ 0.3529, -0.4676, -0.8678,  ...,  0.4083,  1.2413, -0.7714],
        [ 0.3496, -0.4618, -0.8693,  ...,  0.4074,  1.2379, -0.7640],
        [ 0.3485, -0.4610, -0.8667,  ...,  0.4097,  1.2387, -0.7629],
        [ 0.3577, -0.4467, -0.8538,  ...,  0.4053,  1.2148, -0.7572]],
       device='cuda:0', grad_fn=<AddmmBackward0>)
```

record of GPT-2 parameters

In [86]:
```
for name, param in model.named_parameters():
    print(name, param.requires_grad)
```

```
emb.embedding.weight True
layers.0.mha.qkv.weight True
layers.0.mha.qkv.bias True
layers.0.mha.out.weight True
layers.0.mha.out.bias True
layers.0.ff.output_linear.0.weight True
layers.0.ff.output_linear.0.bias True
layers.0.ff.output_linear.2.weight True
layers.0.ff.output_linear.2.bias True
layers.0.attn_norm.norm.weight True
layers.0.attn_norm.norm.bias True
layers.0.ff_norm.norm.weight True
layers.0.ff_norm.norm.bias True
layers.1.mha.qkv.weight True
layers.1.mha.qkv.bias True
layers.1.mha.out.weight True
layers.1.mha.out.bias True
layers.1.ff.output_linear.0.weight True
layers.1.ff.output_linear.0.bias True
layers.1.ff.output_linear.2.weight True
layers.1.ff.output_linear.2.bias True
layers.1.attn_norm.norm.weight True
layers.1.attn_norm.norm.bias True
layers.1.ff_norm.norm.weight True
layers.1.ff_norm.norm.bias True
layers.2.mha.qkv.weight True
layers.2.mha.qkv.bias True
layers.2.mha.out.weight True
layers.2.mha.out.bias True
layers.2.ff.output_linear.0.weight True
layers.2.ff.output_linear.0.bias True
layers.2.ff.output_linear.2.weight True
layers.2.ff.output_linear.2.bias True
layers.2.attn_norm.norm.weight True
layers.2.attn_norm.norm.bias True
layers.2.ff_norm.norm.weight True
layers.2.ff_norm.norm.bias True
layers.3.mha.qkv.weight True
layers.3.mha.qkv.bias True
layers.3.mha.out.weight True
layers.3.mha.out.bias True
layers.3.ff.output_linear.0.weight True
layers.3.ff.output_linear.0.bias True
layers.3.ff.output_linear.2.weight True
layers.3.ff.output_linear.2.bias True
layers.3.attn_norm.norm.weight True
layers.3.attn_norm.norm.bias True
layers.3.ff_norm.norm.weight True
layers.3.ff_norm.norm.bias True
layers.4.mha.qkv.weight True
layers.4.mha.qkv.bias True
layers.4.mha.out.weight True
layers.4.mha.out.bias True
layers.4.ff.output_linear.0.weight True
layers.4.ff.output_linear.0.bias True
layers.4.ff.output_linear.2.weight True
```

layers.4.ff.output_linear.2.bias True
layers.4.attn_norm.norm.weight True
layers.4.attn_norm.norm.bias True
layers.4.ff_norm.norm.weight True
layers.4.ff_norm.norm.bias True
layers.5.mha.qkv.weight True
layers.5.mha.qkv.bias True
layers.5.mha.out.weight True
layers.5.mha.out.bias True
layers.5.ff.output_linear.0.weight True
layers.5.ff.output_linear.0.bias True
layers.5.ff.output_linear.2.weight True
layers.5.ff.output_linear.2.bias True
layers.5.attn_norm.norm.weight True
layers.5.attn_norm.norm.bias True
layers.5.ff_norm.norm.weight True
layers.5.ff_norm.norm.bias True
layers.6.mha.qkv.weight True
layers.6.mha.qkv.bias True
layers.6.mha.out.weight True
layers.6.mha.out.bias True
layers.6.ff.output_linear.0.weight True
layers.6.ff.output_linear.0.bias True
layers.6.ff.output_linear.2.weight True
layers.6.ff.output_linear.2.bias True
layers.6.attn_norm.norm.weight True
layers.6.attn_norm.norm.bias True
layers.6.ff_norm.norm.weight True
layers.6.ff_norm.norm.bias True
layers.7.mha.qkv.weight True
layers.7.mha.qkv.bias True
layers.7.mha.out.weight True
layers.7.mha.out.bias True
layers.7.ff.output_linear.0.weight True
layers.7.ff.output_linear.0.bias True
layers.7.ff.output_linear.2.weight True
layers.7.ff.output_linear.2.bias True
layers.7.attn_norm.norm.weight True
layers.7.attn_norm.norm.bias True
layers.7.ff_norm.norm.weight True
layers.7.ff_norm.norm.bias True
layers.8.mha.qkv.weight True
layers.8.mha.qkv.bias True
layers.8.mha.out.weight True
layers.8.mha.out.bias True
layers.8.ff.output_linear.0.weight True
layers.8.ff.output_linear.0.bias True
layers.8.ff.output_linear.2.weight True
layers.8.ff.output_linear.2.bias True
layers.8.attn_norm.norm.weight True
layers.8.attn_norm.norm.bias True
layers.8.ff_norm.norm.weight True
layers.8.ff_norm.norm.bias True
layers.9.mha.qkv.weight True
layers.9.mha.qkv.bias True
layers.9.mha.out.weight True

```
layers.9.mha.out.bias True
layers.9.ff.output_linear.0.weight True
layers.9.ff.output_linear.0.bias True
layers.9.ff.output_linear.2.weight True
layers.9.ff.output_linear.2.bias True
layers.9.attn_norm.norm.weight True
layers.9.attn_norm.norm.bias True
layers.9.ff_norm.norm.weight True
layers.9.ff_norm.norm.bias True
layers.10.mha.qkv.weight True
layers.10.mha.qkv.bias True
layers.10.mha.out.weight True
layers.10.mha.out.bias True
layers.10.ff.output_linear.0.weight True
layers.10.ff.output_linear.0.bias True
layers.10.ff.output_linear.2.weight True
layers.10.ff.output_linear.2.bias True
layers.10.attn_norm.norm.weight True
layers.10.attn_norm.norm.bias True
layers.10.ff_norm.norm.weight True
layers.10.ff_norm.norm.bias True
layers.11.mha.qkv.weight True
layers.11.mha.qkv.bias True
layers.11.mha.out.weight True
layers.11.mha.out.bias True
layers.11.ff.output_linear.0.weight True
layers.11.ff.output_linear.0.bias True
layers.11.ff.output_linear.2.weight True
layers.11.ff.output_linear.2.bias True
layers.11.attn_norm.norm.weight True
layers.11.attn_norm.norm.bias True
layers.11.ff_norm.norm.weight True
layers.11.ff_norm.norm.bias True
```

In [87]:
```
total_params = sum(p.numel() for p in model.parameters())
print(total_params)
```

```
96109824
```

In [88]:
```
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(trainable_params)
```

```
96109824
```

In [89]:
```
num_param_tensors = len(list(model.named_parameters()))
print(num_param_tensors)
```

```
145
```

In [90]:
```
all_trainable = all(p.requires_grad for p in model.parameters())
print("All parameters trainable:", all_trainable)
```

```
All parameters trainable: True
```

MY GPT2

In [91]:
```
class MY_GPT(nn.Module):
    def __init__(self, embdim,vocab_size):
        super().__init__()
        self.out_layer=GPT2OutputHead(768,vocab_size)
        self.model=GPT2(vocab_size=vocab_size)

    def forward(self, x):
        x=self.model(x)
        x=self.out_layer(x)
        return x
```

In [92]:
```
my_model=MY_GPT(768,vocab_size).to(device)
```

In [93]:
```
my_model
```

Out[93]:
```
MY_GPT(
  (out_layer): GPT2OutputHead(
    (pool): AttentionPooling(
      (score): Linear(in_features=768, out_features=1, bias=True)
    )
    (fc): Linear(in_features=768, out_features=38987, bias=True)
  )
  (model): GPT2(
    (emb): embpos(
      (embedding): Embedding(38987, 768)
    )
    (layers): ModuleList(
      (0-11): 12 x DecoderBlock(
        (mha): MultiHeadAttention(
          (qkv): Linear(in_features=768, out_features=2304, bias=True)
          (out): Linear(in_features=768, out_features=768, bias=True)
        )
        (ff): FeedForward(
          (output_linear): Sequential(
            (0): Linear(in_features=768, out_features=2048, bias=True)
            (1): GELU(approximate='none')
            (2): Linear(in_features=2048, out_features=768, bias=True)
          )
        )
        (attn_norm): AddResidual_LayerNorm(
          (norm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
        )
        (ff_norm): AddResidual_LayerNorm(
          (norm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
        )
      )
    )
  )
)
```

record of MY-MODEL GPT-2 parameters

In [94]:
```
for name, param in my_model.named_parameters():
    print(name, param.requires_grad)
```

```
out_layer.pool.score.weight True
out_layer.pool.score.bias True
out_layer.fc.weight True
out_layer.fc.bias True
model.emb.embedding.weight True
model.layers.0.mha.qkv.weight True
model.layers.0.mha.qkv.bias True
model.layers.0.mha.out.weight True
model.layers.0.mha.out.bias True
model.layers.0.ff.output_linear.0.weight True
model.layers.0.ff.output_linear.0.bias True
model.layers.0.ff.output_linear.2.weight True
model.layers.0.ff.output_linear.2.bias True
model.layers.0.attn_norm.norm.weight True
model.layers.0.attn_norm.norm.bias True
model.layers.0.ff_norm.norm.weight True
model.layers.0.ff_norm.norm.bias True
model.layers.1.mha.qkv.weight True
model.layers.1.mha.qkv.bias True
model.layers.1.mha.out.weight True
model.layers.1.mha.out.bias True
model.layers.1.ff.output_linear.0.weight True
model.layers.1.ff.output_linear.0.bias True
model.layers.1.ff.output_linear.2.weight True
model.layers.1.ff.output_linear.2.bias True
model.layers.1.attn_norm.norm.weight True
model.layers.1.attn_norm.norm.bias True
model.layers.1.ff_norm.norm.weight True
model.layers.1.ff_norm.norm.bias True
model.layers.2.mha.qkv.weight True
model.layers.2.mha.qkv.bias True
model.layers.2.mha.out.weight True
model.layers.2.mha.out.bias True
model.layers.2.ff.output_linear.0.weight True
model.layers.2.ff.output_linear.0.bias True
model.layers.2.ff.output_linear.2.weight True
model.layers.2.ff.output_linear.2.bias True
model.layers.2.attn_norm.norm.weight True
model.layers.2.attn_norm.norm.bias True
model.layers.2.ff_norm.norm.weight True
model.layers.2.ff_norm.norm.bias True
model.layers.3.mha.qkv.weight True
model.layers.3.mha.qkv.bias True
model.layers.3.mha.out.weight True
model.layers.3.mha.out.bias True
model.layers.3.ff.output_linear.0.weight True
model.layers.3.ff.output_linear.0.bias True
model.layers.3.ff.output_linear.2.weight True
model.layers.3.ff.output_linear.2.bias True
model.layers.3.attn_norm.norm.weight True
model.layers.3.attn_norm.norm.bias True
model.layers.3.ff_norm.norm.weight True
model.layers.3.ff_norm.norm.bias True
model.layers.4.mha.qkv.weight True
model.layers.4.mha.qkv.bias True
model.layers.4.mha.out.weight True
```

model.layers.4.mha.out.bias True
model.layers.4.ff.output_linear.0.weight True
model.layers.4.ff.output_linear.0.bias True
model.layers.4.ff.output_linear.2.weight True
model.layers.4.ff.output_linear.2.bias True
model.layers.4.attn_norm.norm.weight True
model.layers.4.attn_norm.norm.bias True
model.layers.4.ff_norm.norm.weight True
model.layers.4.ff_norm.norm.bias True
model.layers.5.mha.qkv.weight True
model.layers.5.mha.qkv.bias True
model.layers.5.mha.out.weight True
model.layers.5.mha.out.bias True
model.layers.5.ff.output_linear.0.weight True
model.layers.5.ff.output_linear.0.bias True
model.layers.5.ff.output_linear.2.weight True
model.layers.5.ff.output_linear.2.bias True
model.layers.5.attn_norm.norm.weight True
model.layers.5.attn_norm.norm.bias True
model.layers.5.ff_norm.norm.weight True
model.layers.5.ff_norm.norm.bias True
model.layers.6.mha.qkv.weight True
model.layers.6.mha.qkv.bias True
model.layers.6.mha.out.weight True
model.layers.6.mha.out.bias True
model.layers.6.ff.output_linear.0.weight True
model.layers.6.ff.output_linear.0.bias True
model.layers.6.ff.output_linear.2.weight True
model.layers.6.ff.output_linear.2.bias True
model.layers.6.attn_norm.norm.weight True
model.layers.6.attn_norm.norm.bias True
model.layers.6.ff_norm.norm.weight True
model.layers.6.ff_norm.norm.bias True
model.layers.7.mha.qkv.weight True
model.layers.7.mha.qkv.bias True
model.layers.7.mha.out.weight True
model.layers.7.mha.out.bias True
model.layers.7.ff.output_linear.0.weight True
model.layers.7.ff.output_linear.0.bias True
model.layers.7.ff.output_linear.2.weight True
model.layers.7.ff.output_linear.2.bias True
model.layers.7.attn_norm.norm.weight True
model.layers.7.attn_norm.norm.bias True
model.layers.7.ff_norm.norm.weight True
model.layers.7.ff_norm.norm.bias True
model.layers.8.mha.qkv.weight True
model.layers.8.mha.qkv.bias True
model.layers.8.mha.out.weight True
model.layers.8.mha.out.bias True
model.layers.8.ff.output_linear.0.weight True
model.layers.8.ff.output_linear.0.bias True
model.layers.8.ff.output_linear.2.weight True
model.layers.8.ff.output_linear.2.bias True
model.layers.8.attn_norm.norm.weight True
model.layers.8.attn_norm.norm.bias True
model.layers.8.ff_norm.norm.weight True

```
model.layers.8.ff_norm.norm.bias True
model.layers.9.mha.qkv.weight True
model.layers.9.mha.qkv.bias True
model.layers.9.mha.out.weight True
model.layers.9.mha.out.bias True
model.layers.9.ff.output_linear.0.weight True
model.layers.9.ff.output_linear.0.bias True
model.layers.9.ff.output_linear.2.weight True
model.layers.9.ff.output_linear.2.bias True
model.layers.9.attn_norm.norm.weight True
model.layers.9.attn_norm.norm.bias True
model.layers.9.ff_norm.norm.weight True
model.layers.9.ff_norm.norm.bias True
model.layers.10.mha.qkv.weight True
model.layers.10.mha.qkv.bias True
model.layers.10.mha.out.weight True
model.layers.10.mha.out.bias True
model.layers.10.ff.output_linear.0.weight True
model.layers.10.ff.output_linear.0.bias True
model.layers.10.ff.output_linear.2.weight True
model.layers.10.ff.output_linear.2.bias True
model.layers.10.attn_norm.norm.weight True
model.layers.10.attn_norm.norm.bias True
model.layers.10.ff_norm.norm.weight True
model.layers.10.ff_norm.norm.bias True
model.layers.11.mha.qkv.weight True
model.layers.11.mha.qkv.bias True
model.layers.11.mha.out.weight True
model.layers.11.mha.out.bias True
model.layers.11.ff.output_linear.0.weight True
model.layers.11.ff.output_linear.0.bias True
model.layers.11.ff.output_linear.2.weight True
model.layers.11.ff.output_linear.2.bias True
model.layers.11.attn_norm.norm.weight True
model.layers.11.attn_norm.norm.bias True
model.layers.11.ff_norm.norm.weight True
model.layers.11.ff_norm.norm.bias True
```

In [95]:
```
trainable_params = sum(p.numel() for p in my_model.parameters() if p.requires_grad)
print(trainable_params)
```

```
126091596
```

In [96]:
```
num_param_tensors = len(list(my_model.named_parameters()))
print(num_param_tensors)
```

```
149
```

In [97]:
```
all_trainable = all(p.requires_grad for p in my_model.parameters())
print("All parameters trainable:", all_trainable)
```

```
All parameters trainable: True
```

training

In [98]:
```python
def accuracy_fn(logits, y):
    preds = torch.argmax(logits, dim=-1)  # (B,)
    y = y.view(-1)                        # (B,)
    correct = (preds == y).float()
    return correct.mean().item()
```

In [99]:
```python
def train(model,EPOCH, batch_size):
    my_model.train()

    for epoch in range(EPOCH):
        total_loss = 0.0
        total_acc = 0.0

        for step, (x, y) in enumerate(dataloader):
            x = x.to(device)   # (B, T)
            y = y.to(device)   # (B, T)
            y = y.long()

            optimizer.zero_grad()

            # forward
            logits = my_model(x)

            # loss
            loss = loss_fn(logits, y)

            # backward
            loss.backward()
            optimizer.step()

            # accuracy
            preds = logits.argmax(dim=-1)  # (B)
            acc = accuracy_fn(logits, y)

            total_loss += loss.item()
            total_acc += acc

            # batch progress
            if (step + 1) % batch_size == 0:
                print(f"Epoch: {epoch+1}/{EPOCH} | Step: {step+1}/{len(dataloader)}

        avg_loss = total_loss / len(dataloader)
        avg_acc = total_acc / len(dataloader)

        print(f"\nEpoch: {epoch+1} | Avg Loss: {avg_loss:.4f} | Avg Acc: {avg_acc:.

    return avg_loss, avg_acc
```

In [100…
```python
optimizer = torch.optim.Adam(my_model.parameters(), lr=6e-4)
loss_fn = nn.CrossEntropyLoss()
```

In [101…
```python
%%time
train(my_model,EPOCH=1, batch_size=4)
```

```
Epoch: 1/1 | Step: 4/259130 | Loss: 41.5310 | Acc: 0.0000
Epoch: 1/1 | Step: 8/259130 | Loss: 9.9886 | Acc: 0.0000
Epoch: 1/1 | Step: 12/259130 | Loss: 16.0300 | Acc: 0.0000
Epoch: 1/1 | Step: 16/259130 | Loss: 18.4975 | Acc: 0.0000
Epoch: 1/1 | Step: 20/259130 | Loss: 12.6972 | Acc: 0.0000
Epoch: 1/1 | Step: 24/259130 | Loss: 17.6402 | Acc: 0.0000
Epoch: 1/1 | Step: 28/259130 | Loss: 16.7050 | Acc: 0.0000
Epoch: 1/1 | Step: 32/259130 | Loss: 20.4177 | Acc: 0.0000
Epoch: 1/1 | Step: 36/259130 | Loss: 14.5927 | Acc: 0.0000
Epoch: 1/1 | Step: 40/259130 | Loss: 16.7949 | Acc: 0.0000
Epoch: 1/1 | Step: 44/259130 | Loss: 19.1036 | Acc: 0.0000
Epoch: 1/1 | Step: 48/259130 | Loss: 10.8507 | Acc: 0.0000
Epoch: 1/1 | Step: 52/259130 | Loss: 12.7809 | Acc: 0.0000
Epoch: 1/1 | Step: 56/259130 | Loss: 15.1148 | Acc: 0.0000
Epoch: 1/1 | Step: 60/259130 | Loss: 11.8238 | Acc: 0.0000
Epoch: 1/1 | Step: 64/259130 | Loss: 16.2286 | Acc: 0.0000
Epoch: 1/1 | Step: 68/259130 | Loss: 12.6338 | Acc: 0.0000
Epoch: 1/1 | Step: 72/259130 | Loss: 11.6361 | Acc: 0.0000
Epoch: 1/1 | Step: 76/259130 | Loss: 6.9507 | Acc: 0.0000
Epoch: 1/1 | Step: 80/259130 | Loss: 11.5652 | Acc: 0.0000
Epoch: 1/1 | Step: 84/259130 | Loss: 7.5010 | Acc: 0.2500
Epoch: 1/1 | Step: 88/259130 | Loss: 6.8747 | Acc: 0.0000
Epoch: 1/1 | Step: 92/259130 | Loss: 13.5695 | Acc: 0.0000
Epoch: 1/1 | Step: 96/259130 | Loss: 7.3027 | Acc: 0.0000
Epoch: 1/1 | Step: 100/259130 | Loss: 12.9339 | Acc: 0.0000
Epoch: 1/1 | Step: 104/259130 | Loss: 13.2563 | Acc: 0.0000
Epoch: 1/1 | Step: 108/259130 | Loss: 7.0772 | Acc: 0.0000
Epoch: 1/1 | Step: 112/259130 | Loss: 23.3429 | Acc: 0.0000
Epoch: 1/1 | Step: 116/259130 | Loss: 16.8295 | Acc: 0.0000
Epoch: 1/1 | Step: 120/259130 | Loss: 16.6888 | Acc: 0.0000
Epoch: 1/1 | Step: 124/259130 | Loss: 10.6670 | Acc: 0.0000
Epoch: 1/1 | Step: 128/259130 | Loss: 11.7812 | Acc: 0.0000
Epoch: 1/1 | Step: 132/259130 | Loss: 15.2022 | Acc: 0.2500
Epoch: 1/1 | Step: 136/259130 | Loss: 16.8442 | Acc: 0.0000
Epoch: 1/1 | Step: 140/259130 | Loss: 27.8420 | Acc: 0.0000
Epoch: 1/1 | Step: 144/259130 | Loss: 20.0009 | Acc: 0.0000
Epoch: 1/1 | Step: 148/259130 | Loss: 14.0645 | Acc: 0.0000
Epoch: 1/1 | Step: 152/259130 | Loss: 10.2819 | Acc: 0.0000
Epoch: 1/1 | Step: 156/259130 | Loss: 11.8593 | Acc: 0.0000
Epoch: 1/1 | Step: 160/259130 | Loss: 14.8405 | Acc: 0.0000
Epoch: 1/1 | Step: 164/259130 | Loss: 21.5850 | Acc: 0.0000
Epoch: 1/1 | Step: 168/259130 | Loss: 11.7095 | Acc: 0.0000
Epoch: 1/1 | Step: 172/259130 | Loss: 11.9486 | Acc: 0.0000
Epoch: 1/1 | Step: 176/259130 | Loss: 11.2229 | Acc: 0.0000
Epoch: 1/1 | Step: 180/259130 | Loss: 10.8243 | Acc: 0.0000
Epoch: 1/1 | Step: 184/259130 | Loss: 15.3466 | Acc: 0.0000
Epoch: 1/1 | Step: 188/259130 | Loss: 9.9233 | Acc: 0.0000
Epoch: 1/1 | Step: 192/259130 | Loss: 9.1795 | Acc: 0.0000
Epoch: 1/1 | Step: 196/259130 | Loss: 10.5165 | Acc: 0.0000
KeyboardInterrupt
```

In [102…

```python
from pathlib import Path
root=Path('models')
root.mkdir(exist_ok=True)
path1= root / "GPT.pth"
```

```
path2= root / "GPT_outlayer.pth"
path3= root / "MY_GPT.pth"
```

In [103…
```
torch.save(model,path1)
```

In [104…
```
torch.save(out_layer,path2)
```

In [105…
```
torch.save(my_model,path3)
```

In [95]:
```
!cd F:\Projects\Transfomers
```

In [ ]:
```
!jupyter nbconvert --to webpdf GPT-2shortent.ipynb
```

In [ ]:
```

```